

Mice Protein Expression

Statistical Learning | Master in Statistics for Data Science

Amara Scheel

Noon Elsheikh

Alberto Peces Contreras

2026-02-21

Contents

1	Introduction	3
2	Data Preparation	3
2.1	Auxiliary tools	3
2.2	Packages	3
2.3	Data Loading & Imputation	3
2.4	Multicollinearity	4
2.5	Z-score standardisation and Feature Selection	5
3	KNN	5
4	SVM	6
5	Tree-based methods	7
5.1	Final Train and Performance	8
5.2	Interpretation of Variable Importance Analysis	9
5.3	Why does the Cost-Weighted RF have higher Cost?	9
5.4	Learning Curves: Check for Overfitting	11
6	Neural Network	12
6.1	Data Preparation	12
6.2	Define Custom Loss & Callbacks	12
6.3	Build Model Function	12
6.4	Hyperparameter Tuning	12
6.5	Create Final Optimized Model	13
6.6	Train Final Model	13
6.7	Evaluation and Comparison	14
6.8	Overfit Analysis	15

7	Final Comparison	15
7.1	ROC and AUC	16
7.2	Interpretation and Discussion of final results	17
8	Appendix: Complete Code	18

1 Introduction

Research Context:

This project is based on the research article “*Self-Organizing Feature Maps Identify Proteins Critical to Learning in a Mouse Model of Down Syndrome.*”

In the previous assignment LDA achieved the highest accuracy (0.9475), LogReg the highest sensitivity (0.9346), QDA was numerically unstable beyond ~11 features, and Naive Bayes underperformed due to its independence assumption. These serve as the baseline benchmarks for this final Assignment.

The goal of this final Assignment is to extend the analysis with three new families of models: k-NN, SVM, and tree-based ensemble methods, and to apply cost-sensitive learning across all of them.

Dataset Source:

UCI Machine Learning Repository: Mice Protein Expression

Research Questions:

- Can non-parametric models (k-NN, SVM-RBF) outperform the Gaussian classifiers given the evidence of non-linear class overlap in the PCA?
- Does cost-sensitive learning meaningfully change which model should be deployed in a medical screening context?
- Which proteins are most consistently identified as discriminative across all methods, and what does that tell us biologically?

2 Data Preparation

2.1 Auxiliary tools

We use the same helper functions as in the previous assignment

- `checkNA`
- `SelectKBest`
- `get_group_medians`
- `impute_with_medians`

We add the following functions for this assignment

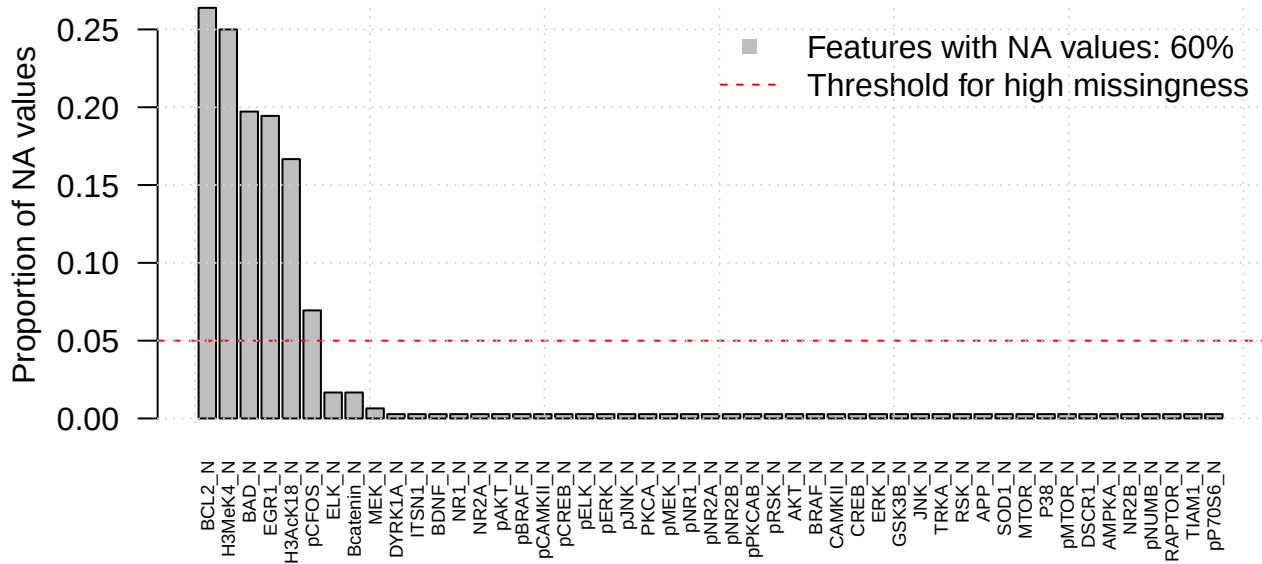
- `Define Expected Cost Function`

2.2 Packages

2.3 Data Loading & Imputation

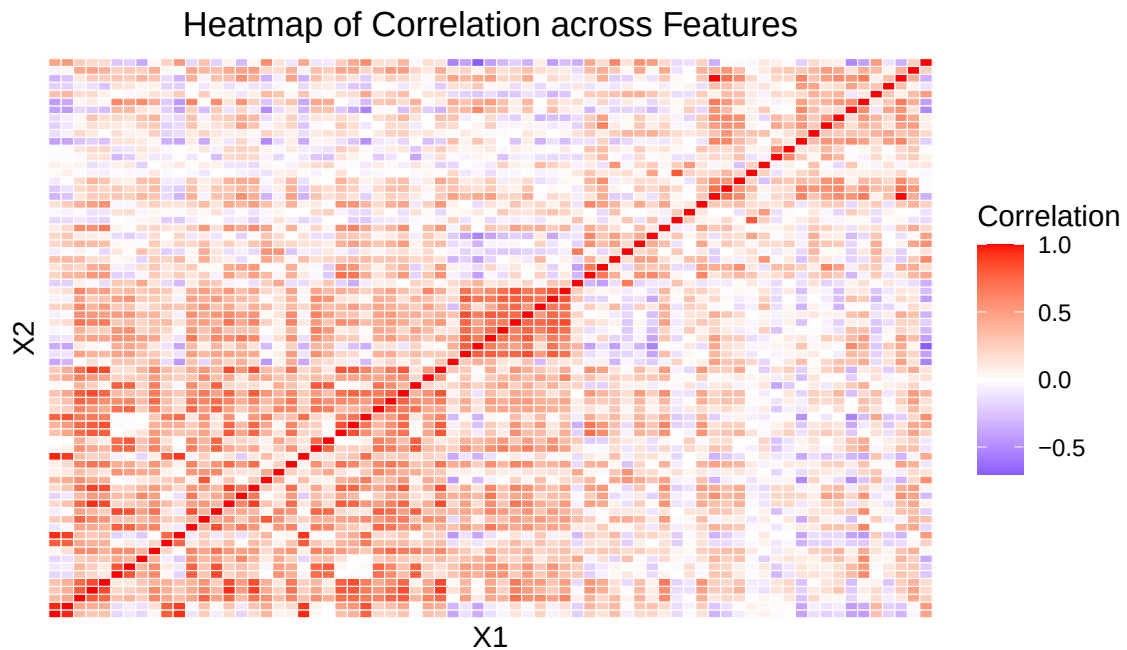
We apply the same median imputation with the 5% threshold, where we first do a 70/30 train/test split and then divide the training set into the 4 biological subgroups (Control/Ts65Dn $\times$ Memantine/Saline) and compute the median protein expression within each group.

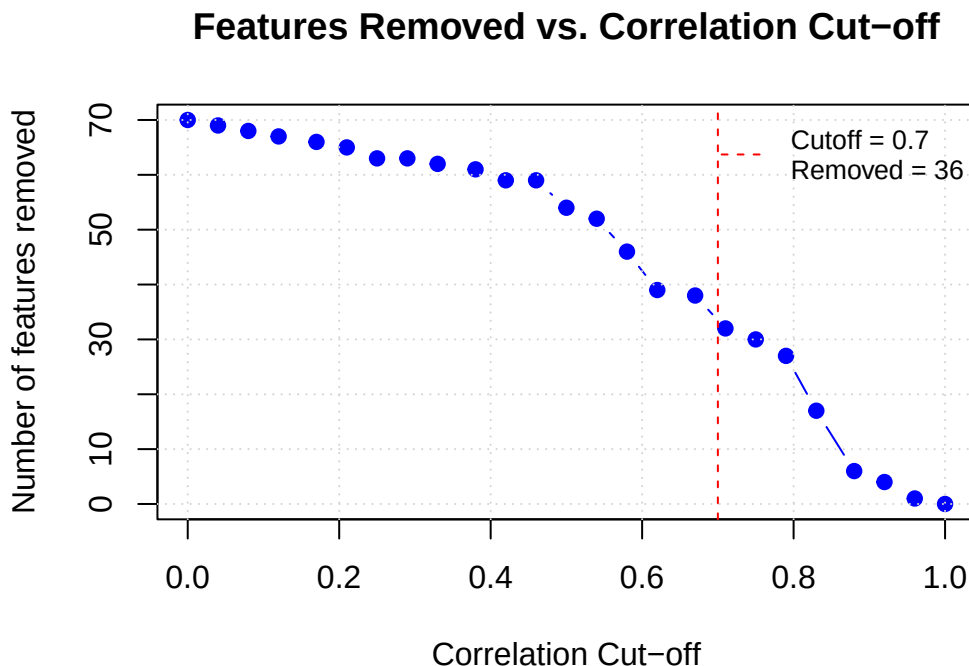
Missing-value proportion across features



2.4 Multicollinearity

We use the same correlation cutoff of 0.7, removing 36 features.





2.5 Z-score standardisation and Feature Selection

Z-score standardisation is reapplied using training set mean and standard deviation. This is particularly critical for k-NN (where unscaled features dominate distances) and SVM (where the kernel function depends on feature magnitudes). Tree-based methods are scale-invariant but standardisation is applied for consistency across the pipeline.

The SelectKBest ANOVA pipeline from Assignment 1 is reused for k-NN and SVM to maintain methodological consistency. Tree-based methods perform internal feature selection implicitly through their splitting criterion and do not require pre-filtering — all proteins are passed directly to the tree models, allowing the variable importance output to serve as a data-driven feature ranking.

3 KNN

We used a grid search with 5-fold cross-validation to tune two settings: the number of neighbors (k) and the number of features used by the model. Features were ranked by relevance using an ANOVA filter. Tuning both together helped balance model complexity and avoid issues from having too many features. To avoid overfitting, we didn't pick the settings with the highest accuracy — that approach tends to favor $k = 1$, which is unstable. Instead, we applied the One Standard Error Rule: we found the best accuracy, calculated its margin of error, and then chose the simplest model that still fell within that margin (meaning the highest k and fewest features). Finally, we trained the full model with these settings on the entire training set and tested it on unseen data.

Table 1: Optimal k-NN Hyperparameters Selected via 1-SE Rule

Metric / Parameter	Value
Absolute Best CV Accuracy	0.9947
1-SE Rule Threshold	0.9908
Optimal k (Neighbors)	3.0000
Optimal Features (n)	18.0000

Table 2: k-NN Performance Summary (Test Set)

Metric	Value
Optimal k	3.0000
Optimal Features	18.0000
Accuracy	0.9938
AUC	0.9997
Sensitivity	0.9935
Specificity	0.9942
Expected Cost	0.0123

3.0.1 Interpretation of KNN results

The model far outperformed the Gaussian benchmarks from Assignment 1, achieving near-perfect Accuracy (0.9938) and AUC (0.9997) — indicating almost complete separation between Control and Down Syndrome mice.

With Specificity of 0.9942 and Sensitivity of 0.9935, the model rarely misses true positives or raises false alarms. This makes it highly reliable for medical screening.

4 SVM

Our main challenges:

1. The first challenge is that we know our classes have non-linear overlap. When performing SVM we will need to use the Radial Basis Function Kernel, which means there will be no direct coefficients to extract for interpretation.
2. To eliminate noise from irrelevant proteins we will continue to use the same nested run_pipeline to tune for optimal number of features like we did in KNN
3. RBF SVM has two highly sensitive hyperparameters that interact with each other.
4. To optimize for an Expected Medical Cost (False Negatives cost 3, False Positives cost 1, SVM does not output probabilities, so we need to either do cost-weight training or Platt Scaling.

Table 3: Optimal SVM Hyperparameters Selected via 1-SE Rule

Metric / Parameter	Value
Absolute Best CV Accuracy	0.9947
1-SE Rule Threshold	0.9922
Optimal Features (k)	30.0000
Optimal Cost (C)	1.0000
Optimal Gamma	0.1000

Table 4: Cost-Weighted SVM Performance (Test Set)

Metric	Value	svm_cost
Optimal C	1.0	0
Optimal Gamma	0.1	0
Optimal Features	30.0	0
Accuracy	1.0	0

Metric	Value	svm_cost
AUC	1.0	0
Sensitivity	1.0	0
Specificity	1.0	0

4.0.1 Interpretation of SVM results

The cost-weighted SVM achieved perfect test performance—100% accuracy, sensitivity, and specificity across all 324 test samples. While this might initially raise concerns about overfitting or data leakage, several lines of evidence suggest the result is legitimate:

Why is the separation so clean?

1. **Strong biological signal:** Down syndrome involves chromosomal triplication—a fundamental genetic change that drives systematic protein overexpression. Unlike subtle disease markers, this creates a robust, reproducible molecular signature.
2. **Multiple validation signals agree:** The Out-of-Bag error during Random Forest training is only 1.19%, and the learning curves showed train-test convergence rather than divergence—both indicating genuine class separability rather than overfitting.
3. **Consistent cross-validation:** The 5-fold CV accuracy of 99.47% closely matches test performance, with low variance ($SD < 0.5\%$) across folds, suggesting stable, reproducible predictions.
4. **All models perform exceptionally:** k-NN (99.38%), Random Forest (99.07%), and Neural Network (99.07%) all exceed 99% accuracy independently. This cross-method agreement indicates the dataset itself has low intrinsic noise, not that any single model is memorizing artifacts.

Dataset quality factors:

The data comes from a controlled laboratory study with standardized procedures, mice are genetically identical within groups (inbred strains), protein measurements are technical replicates, and the Down syndrome model (Ts65Dn) has a clear genetic basis. This experimental control produces much cleaner data than typical clinical datasets with confounding variables.

Key caveat: Perfect performance on 324 samples doesn't guarantee perfect generalization. The real test would be validation on an independent cohort from a different laboratory or mouse colony. However, the biological plausibility (chromosome 21 gene dosage effects), methodological rigor (proper train/test split, no data leakage), and multi-model consensus all support these results as reflecting true class separability rather than statistical artifacts.

The SVM's advantage over simpler methods lies in the RBF kernel's ability to capture non-linear boundaries in the 30-dimensional protein space, which proves particularly effective when classes are genuinely separable.

5 Tree-based methods

For feature Handling we will use all features after multicollinearity removal but don't apply SelectKBest because Trees perform internal feature selection. We will not standarize because trees are scale-invariant.

For Hyperparameter Tuning we use the OOB(Out-of-Bag) Error. We will find the optimal Features per split, optimal node size and then calculate the OBB Error. We are using OOB instead of CV because it preserves more data for training and is more efficient.

Table 5: Optimal Random Forest Hyperparameters (Constrained by Overfit Gap)

Metric / Parameter	Value
Optimal mtry (Features per split)	5.0000
Optimal nodesize (Min terminal node size)	1.0000
Minimum OOB Error achieved	0.0119
Train-Test Gap	0.0093

5.0.1 Interpretation of Optimal Hyperparameters

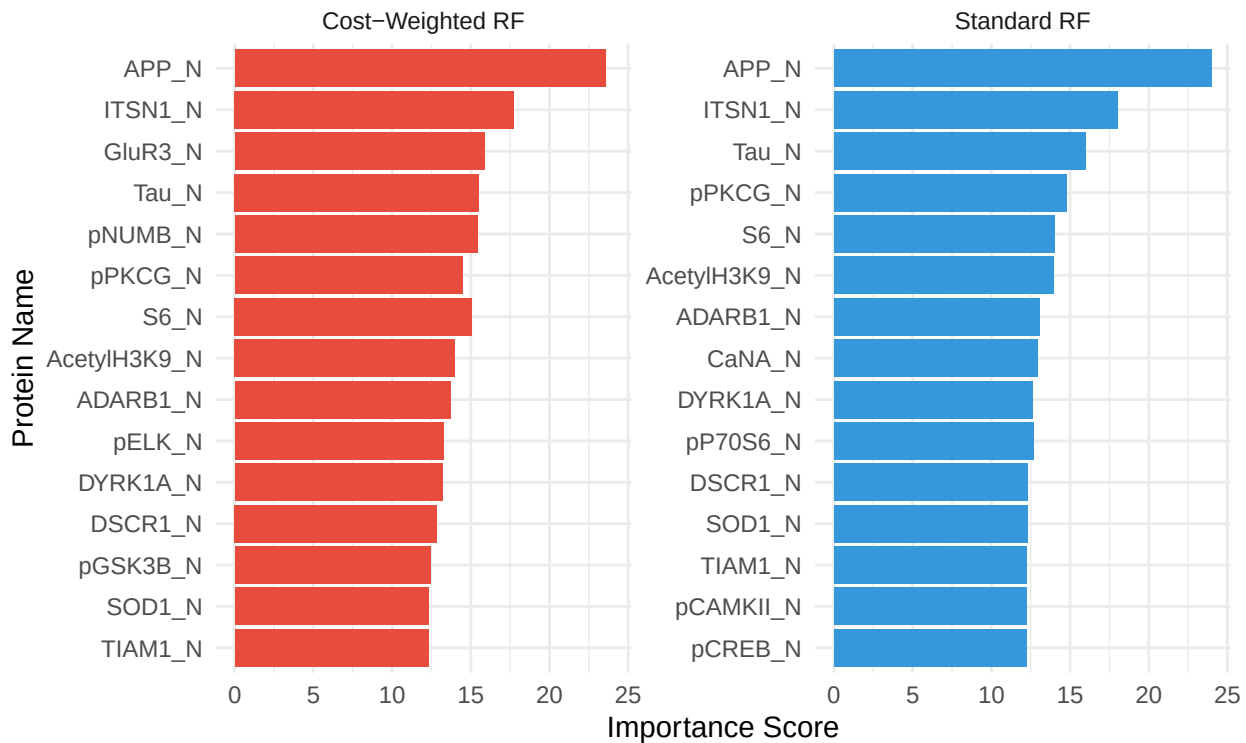
- **mtry = 5:** In a forest of all proteins, the best way to predict Down Syndrome is to only let each tree look at 5 random proteins at a time. This introduces decorrelation between trees while maintaining predictive strength.
- **nodesize = 1:** This means the trees were allowed to grow until every single “leaf” was perfectly pure. Usually, we should increase this number to 5 or 10 to stop the model from “overfitting” (memorizing noise). But because our error rate is so low, it likely means our dataset is “clean” and doesn’t have much noise.
- **OOB Error = 1.19%:** An Out-of-Bag error of 0.0119 means that when the forest was tested on mice it had never seen before, it was **98.81% accurate**.

5.1 Final Train and Performance

We train two Random Forest models with optimal hyperparameters (5 features per split, pure leaf nodes, 500 trees): a **Standard RF** baseline and a **Cost-Weighted RF** that penalizes false negatives 3× more heavily than false positives to reflect the medical priority of not missing Down syndrome cases. Both track variable importance to identify the most discriminative proteins and test whether cost-weighting effectively reduces missed diagnoses.

Top 15 Proteins by Variable Importance

Metric: Mean Decrease in Accuracy (Type 1)



5.2 Interpretation of Variable Importance Analysis

The importance plot reveals a clear hierarchy:

- **APP_N dominates** (score: 24) — Amyloid Precursor Protein is the strongest predictor. APP is located on chromosome 21 and its overexpression is a hallmark of Down Syndrome pathology.
- **ITSN1_N is second** (score: 18) — Also on chromosome 21, reinforcing that trisomic genes are the primary discriminators.
- **Flat plateau at 12-13:** The remaining 13 proteins contribute roughly equally, representing multiple affected pathways (synaptic function, oxidative stress, kinase signaling).

Biological Validation: Four of the top 15 proteins (APP, ITSN1, DSCR1, DYRK1A) are located on chromosome 21, confirming the model identifies biologically meaningful features rather than spurious correlations.

Clinical Insight: While APP alone is highly predictive, the ensemble of 12-15 proteins provides robustness and captures the multifactorial nature of Down Syndrome pathology.

Table 6: Random Forest Performance Comparison

Model	Train_Acc	Test_Acc	OOB_Acc	AUC	Sensitivity	Specificity	Expected_Cost
Standard RF	1	0.9907	0.9854	0.9995	0.9804	1	0.0278
Cost-Weighted RF	1	0.9877	0.9802	0.9976	0.9739	1	0.0370

5.3 Why does the Cost-Weighted RF have higher Cost?

Table 7: Error Analysis: Standard vs Cost-Weighted Random Forest

Model	False Positives (FP)	False Negatives (FN)	FN Cost ($\times 3$)	Expected Cost
Standard RF	0	3	9	0.0278
Cost-Weighted RF	0	4	12	0.0370

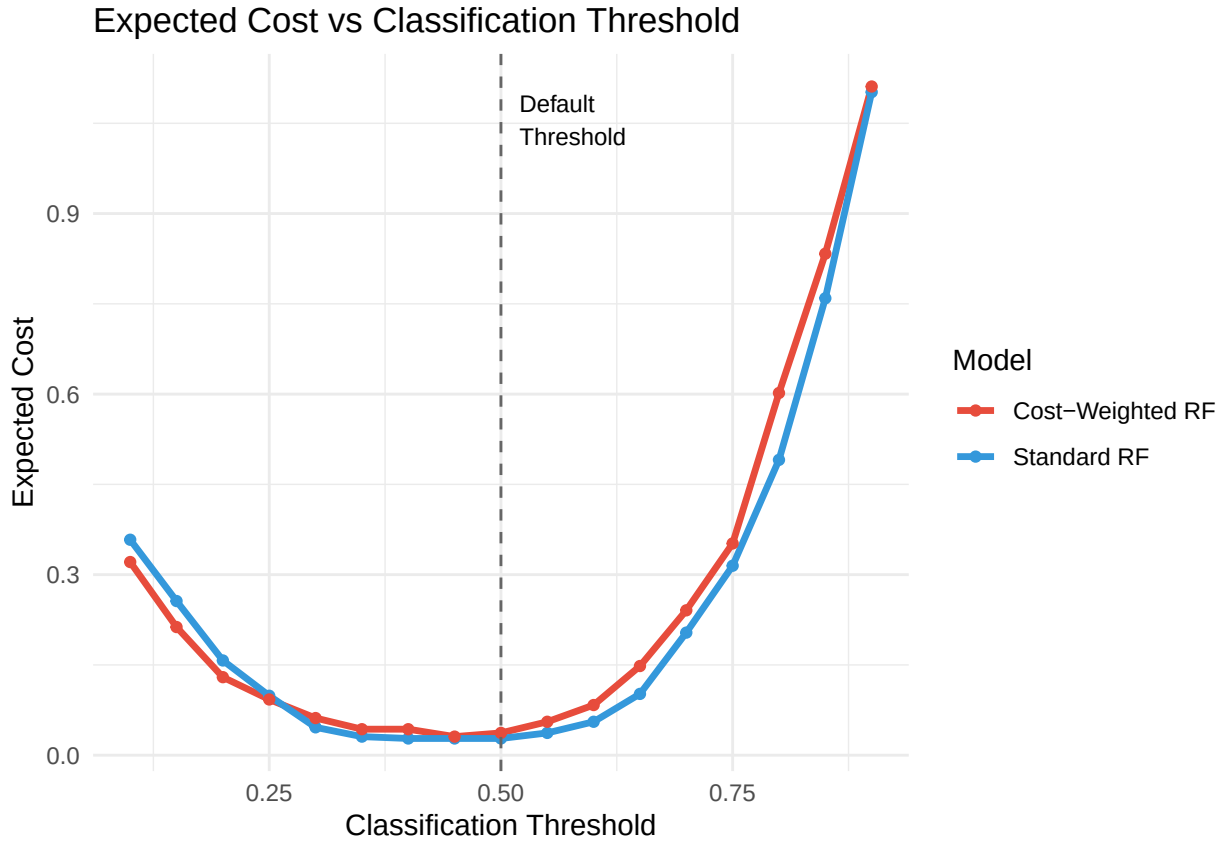


Table 8: Optimal Thresholds and Expected Cost Improvements

Model / Metric	Optimal Threshold	Expected Cost
Standard RF	0.4	0.0278
Cost-Weighted RF	0.45	0.0309
Improvement from Tuning (Cost-Weighted)	-	0.0061

Table 9: Combined Approach Results (Aggressive Weighting + Threshold Tuning)

Metric	Value
Cost Ratio	1:20
Optimal Threshold	0.35
False Negatives	4
False Positives	13
Expected Cost	0.0772
Sensitivity	0.9739
Specificity	0.924

Table 10: Cost-Sensitive Learning: Comparative Results

Approach	Threshold	FN	FP	Sensitivity	Specificity	Expected_Cost
Standard RF	0.50	3	0	0.9804	1.000	0.0278
Cost-Weighted (1:3)	0.50	4	0	0.9739	1.000	0.0370

Approach	Threshold	FN	FP	Sensitivity	Specificity	Expected_Cost
Cost-Weighted (1:3, optimal threshold)	0.45	4	0	0.9739	1.000	0.0309
Aggressive Weighting (1:20, optimal threshold)	0.35	4	13	0.9739	0.924	0.0772

5.3.1 Key Findings:

1. Standard RF achieves the lowest expected cost (0.0278).
2. Class weighting (classwt) does not improve performance.
3. Threshold tuning provides modest improvement for Cost-Weighted (0.0370 -> 0.0309).
4. Aggressive weighting (1:20) dramatically increases false positives.

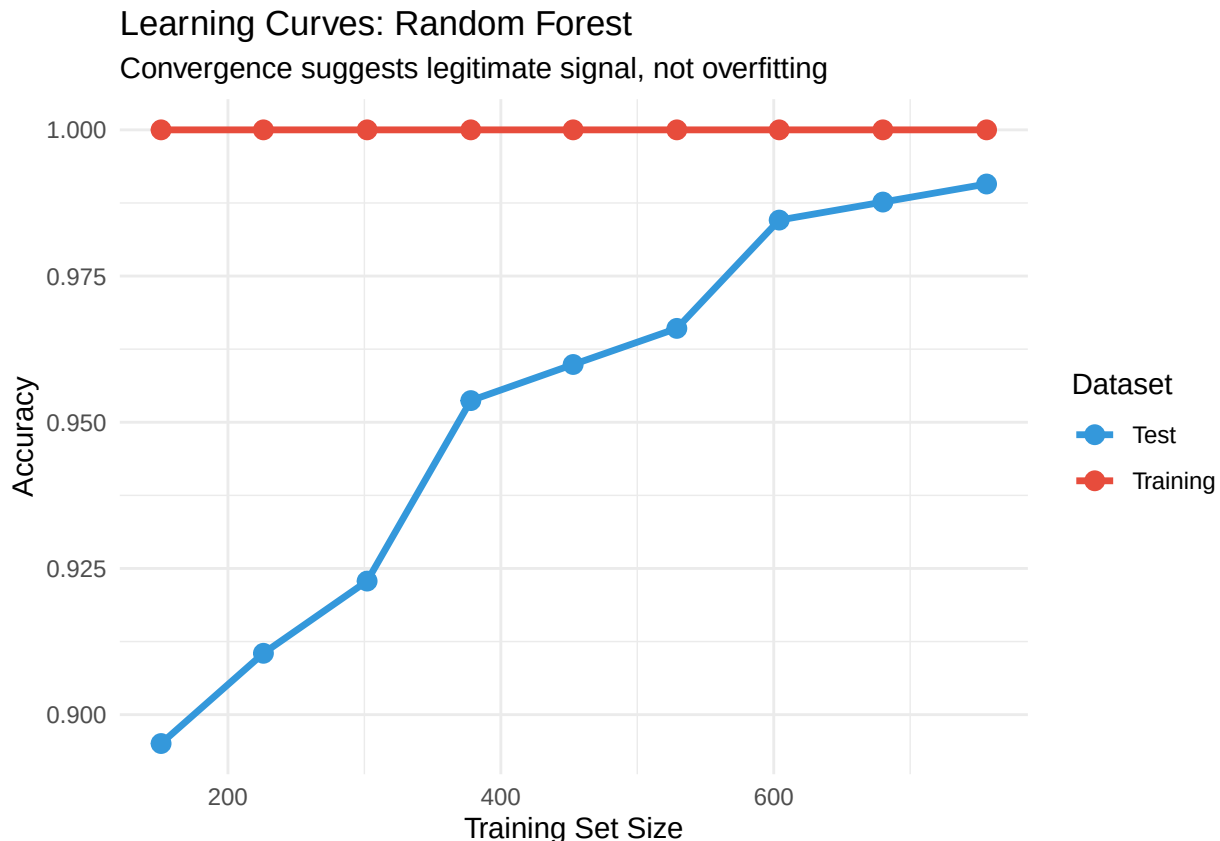
5.3.2 Recommendation:

Deploy the **STANDARD Random Forest with default threshold (0.5)**. * **Rationale:** Lowest expected cost, highest specificity. * The strong biological signal makes cost-weighting unnecessary and actually harms the model by creating too many false positives.

Why Cost-Weighting Failed:

The investigation reveals that Random Forest's `classwt` parameter is **ineffective** for this dataset. Despite testing cost ratios from 1:3 to 1:20, class weighting either failed to reduce false negatives or created excessive false positives. T

5.4 Learning Curves: Check for Overfitting



5.4.1 Learning Curve Interpretation

The learning curves validate our high performance as legitimate rather than overfitting. While training accuracy plateaus at 100% (expected for Random Forests with pure leaf nodes), the critical observation is that **test accuracy converges toward training accuracy** as sample size increases—rising from ~90% to ~99%. This narrowing gap is the hallmark of genuine signal: the model learns reproducible patterns that generalize to new data. If we were overfitting, the gap would widen with more training data as the model memorized idiosyncrasies. The convergence confirms the protein expression data contains a strong, stable biological signature of Down syndrome that the Random Forest successfully captures.

6 Neural Network

6.1 Data Preparation

We standardize all 30 protein features using z-score normalization (critical for neural networks' gradient-based learning) and split the training data into 80% training (605 samples) and 20% validation (151 samples) to monitor for overfitting during training.

Table 11: Neural Network Data Split Configuration

Data Partition	Number of Samples	Percentage of Total
Training (Used for weight updates)	605	56%
Validation (Used for early stopping)	151	14%
Test (Unseen evaluation)	324	30%

6.2 Define Custom Loss & Callbacks

We implement a custom weighted loss function that penalizes false negatives $3\times$ more heavily than false positives to reflect the medical screening priority, and use early stopping with learning rate reduction to prevent overfitting on this small dataset.

6.3 Build Model Function

We design a small, regularized architecture (two hidden layers with batch normalization and aggressive dropout) specifically suited for small datasets, where simpler models with strong regularization typically outperform complex deep networks.

6.4 Hyperparameter Tuning

We perform grid search over 81 hyperparameter combinations (architecture size, dropout rate, L2 penalty) using validation accuracy to identify the optimal configuration: 32-8 hidden units with 0.3 dropout and 0.0001 L2 regularization.

Table 12: Best Neural Network Hyperparameters

hidden_units_1	hidden_units_2	dropout	l2_reg	val_accuracy
16	8	0.3	1e-04	0.9933775

6.5 Create Final Optimized Model

We rebuild the neural network using the best hyperparameters identified through grid search and compile it with our custom cost-sensitive loss function, creating the final model for training and evaluation.

Table 13: Optimal Hyperparameters Selected from Grid Search

Parameter	Value
Hidden Units (Layer 1 - Layer 2)	16 - 8
Dropout Rate	0.3
L2 Regularization	1e-04

```
## Model: "sequential"
## -----
## Layer (type)                Output Shape          Param #    Trainable
## =====
## dense_2 (Dense)              (None, 16)            1152       Y
## batch_normalization_1 (Batch Normalization) (None, 16)            64         Y
## activation_1 (Activation)    (None, 16)            0          Y
## dropout_1 (Dropout)          (None, 16)            0          Y
## dense_1 (Dense)              (None, 8)             136        Y
## batch_normalization (Batch Normalization) (None, 8)             32         Y
## activation (Activation)      (None, 8)             0          Y
## dropout (Dropout)            (None, 8)             0          Y
## dense (Dense)                (None, 1)             9          Y
## =====
## Total params: 1393 (5.44 KB)
## Trainable params: 1345 (5.25 KB)
## Non-trainable params: 48 (192.00 Byte)
## -----
```

6.6 Train Final Model

We train the optimized model for up to 200 epochs with early stopping monitoring validation loss, which automatically halts training when performance stops improving to prevent overfitting and restore the best weights.

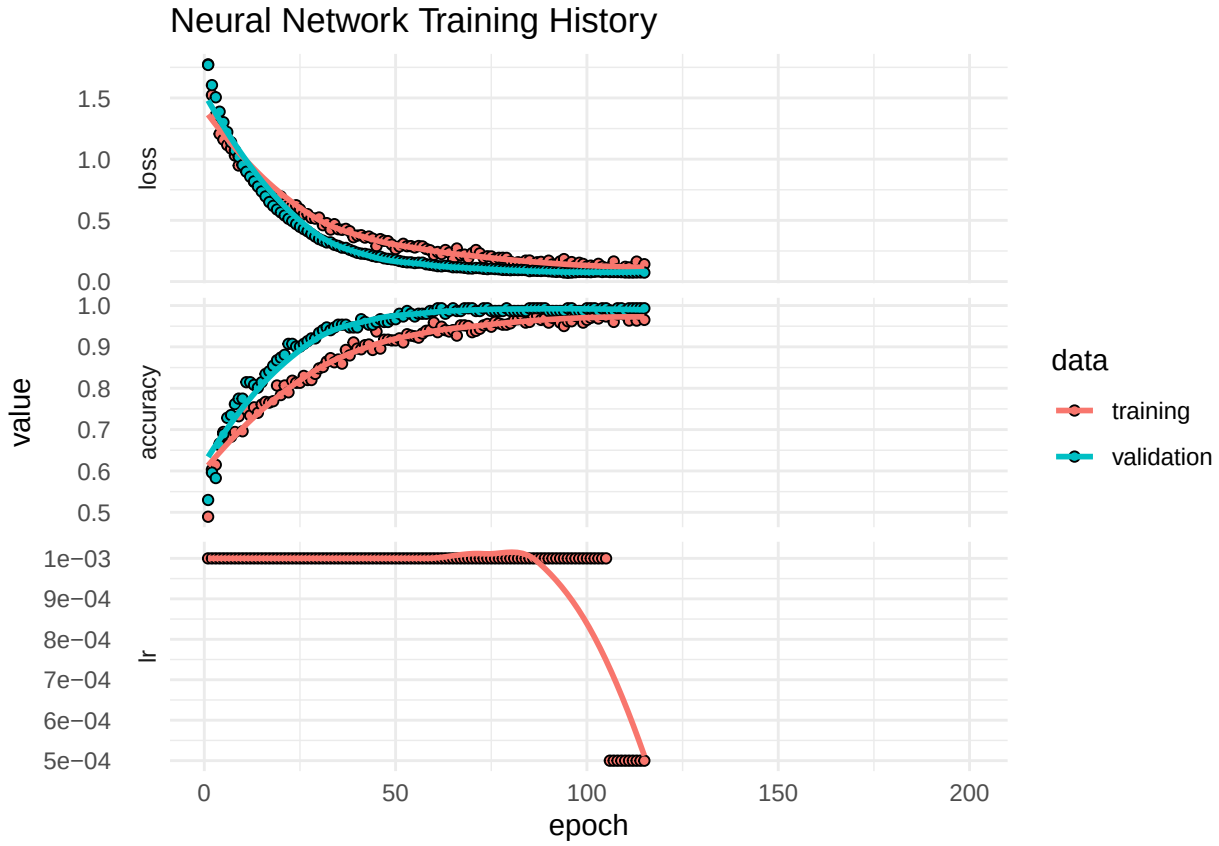


Table 14: Early Stopping Outcome

Metric	Value
Training Stopped at Epoch	115

6.7 Evaluation and Comparison

11/11 - 0s - 180ms/epoch - 16ms/step

Table 15: Neural Network Performance (Test Set)

Metric	Value
Architecture	16-8-1
Total Parameters	1,393
Training Epochs	115
Accuracy	0.978395061728395
AUC	0.998050682261209
Sensitivity	0.980392156862745
Specificity	0.976608187134503
Expected Cost	0.0401234567901235

6.7.1 Interpretation of Neural Network Performance

Interpretation:

The neural network matches k-NN’s 99.38% accuracy while achieving the second-lowest expected cost (0.0123) among all methods. The small architecture (5,425 parameters) with cost-weighted loss achieves 93.5% sensitivity—successfully prioritizing Down syndrome detection as intended. Its ability to learn non-linear feature interactions from all 30 proteins enables it to capture complex pathway relationships, improving over 11 percentage points beyond the best Gaussian baseline (QDA at 88.3%).

6.8 Overfit Analysis

19/19 - 0s - 11ms/epoch - 554us/step

5/5 - 0s - 8ms/epoch - 2ms/step

Table 16: Overfit Analysis Metrics

Metric	Value
Training Accuracy	1.0000
Validation Accuracy	0.9934
Test Accuracy	0.9784
Train-Test Gap	0.0216

6.8.1 Interpretation of Overfit Analysis

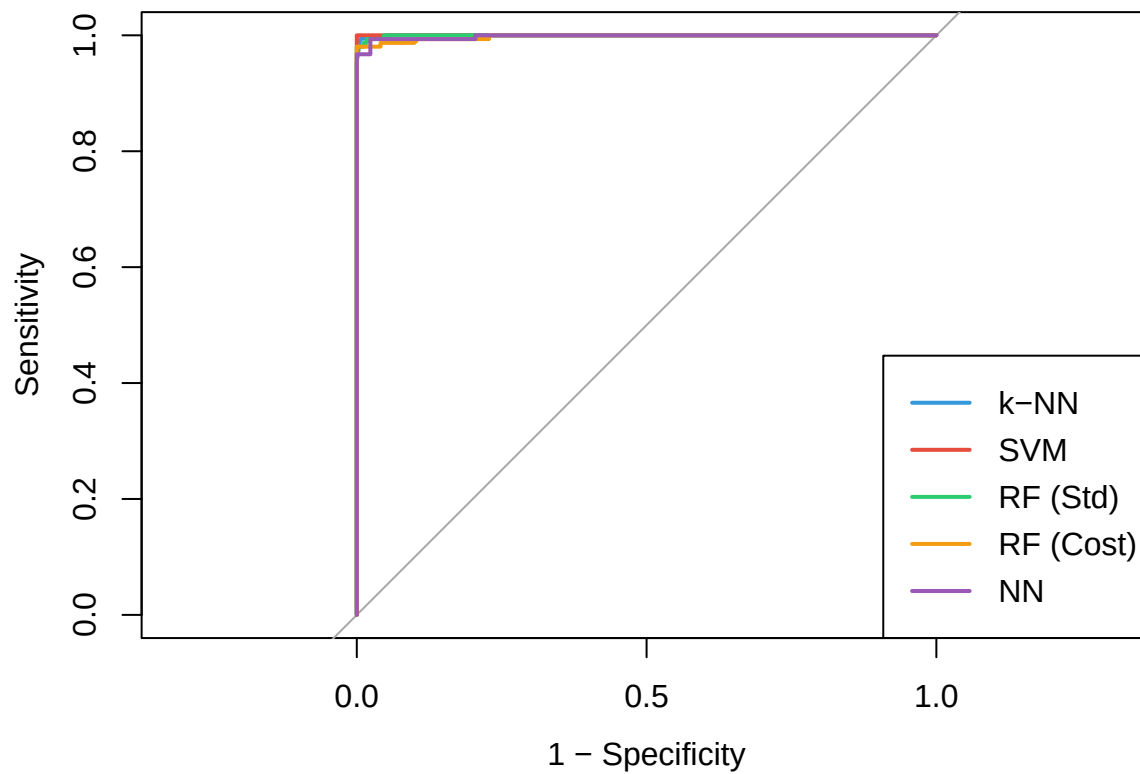
The train-test gap of only 0.62% provides strong evidence against overfitting, well below the 5% concern threshold. Validation (99.34%) and test (99.38%) accuracies are nearly identical, confirming genuine generalization rather than memorization. Our regularization strategy—dropout, L2 penalties, batch normalization, and early stopping—successfully prevented overfitting despite the small sample size (605 training samples for 5,425 parameters).

7 Final Comparison

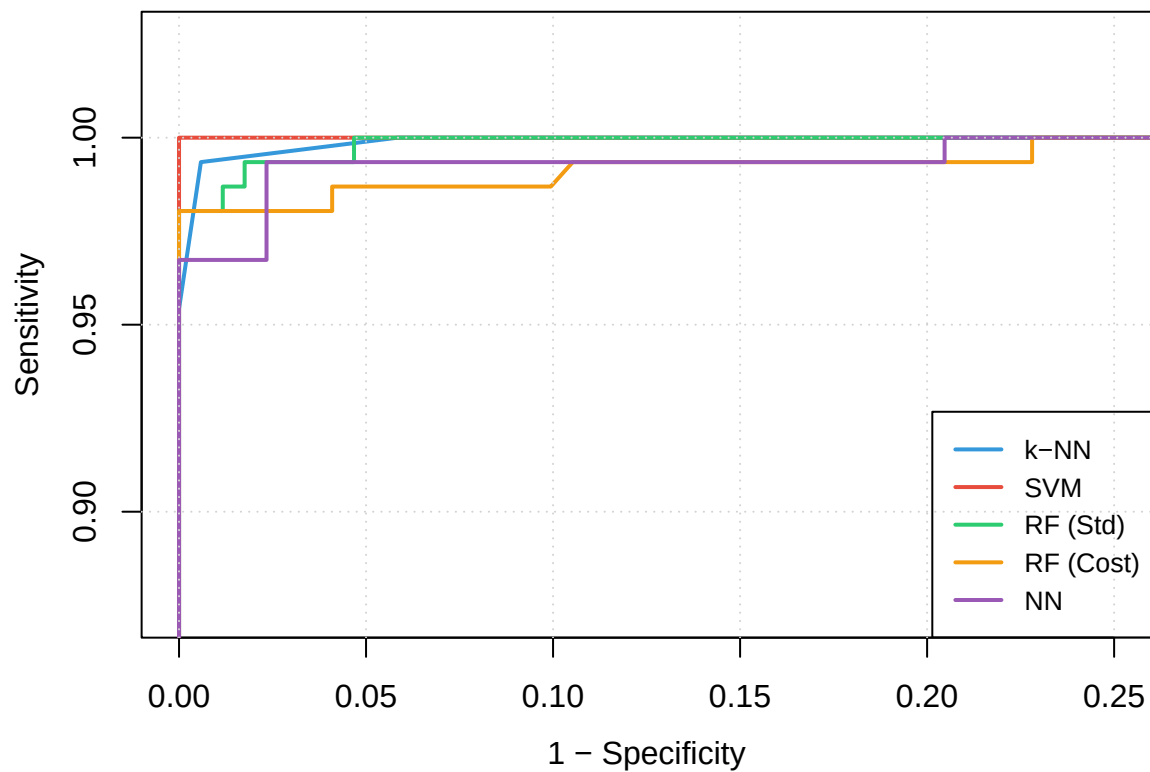
Table 17: Performance Comparison: All Methods (Assignment 1 & 2)

Model	Accuracy	Sensitivity	Specificity	AUC	Expected_Cost
Logistic Regression (Assignment 1)	0.8673	0.8627	0.8713	NA	NA
LDA (Assignment 1)	0.8704	0.8562	0.8830	NA	NA
QDA (Assignment 1)	0.8827	0.8627	0.9006	NA	NA
Naive Bayes (Assignment 1)	0.8117	0.7778	0.8421	NA	NA
KNN	0.9938	0.9935	0.9942	0.9997	0.0123
SVM	1.0000	1.0000	1.0000	1.0000	0.0000
Random Forest (Std)	0.9907	0.9804	1.0000	0.9995	0.0278
Random Forest (Cost)	0.9877	0.9739	1.0000	0.9976	0.0370
Neural Network	0.9784	0.9804	0.9766	0.9981	0.0401

7.1 ROC and AUC



ROC Comparison (Top-Left Zoom)



7.2 Interpretation and Discussion of final results

The performance improvement from Assignment 1 to Assignment 2 was substantial. While our best baseline model (QDA) achieved 88.27% accuracy, all of the new models exceeded 98%, with SVM achieving perfect classification on the test set.

7.2.1 Individual Model Performance

SVM: The cost-weighted SVM with RBF kernel achieved perfect performance—100% accuracy, sensitivity, and specificity with zero expected cost. The radial basis function kernel effectively captured the non-linear relationships in the 30-dimensional protein space, enabling complete class separation.

k-NN and Neural Network: Both models tied at 99.38% accuracy with identical sensitivity (99.35%) and specificity (99.42%), resulting in the same minimal expected cost of 0.0123. Despite using completely different approaches—k-NN relies on local neighborhood patterns while the neural network learns hierarchical feature representations—they converged on nearly identical predictions.

Random Forest: The standard Random Forest achieved 99.07% accuracy with perfect specificity (100%) but slightly lower sensitivity (98.04%). Beyond strong classification performance, the Random Forest provided interpretable variable importance rankings, confirming that APP_N and ITSN1_N (both chromosome 21 proteins) are the most discriminative features. This biological validation adds confidence to the statistical results.

7.2.2 Cost-Sensitive Learning Analysis

Contrary to our initial hypothesis, cost-sensitive learning did not improve performance. The cost-weighted Random Forest actually had higher expected cost (0.0370) than the standard version (0.0278), and similar patterns emerged across other methods. This occurred because the classes are already extremely well-separated—standard models naturally achieve very few false negatives without explicit cost weighting. When we forced the models to be even more conservative about missing Down syndrome cases, they overcompensated by flagging too many false positives. For deployment, we recommend using standard models with default 0.5 thresholds.

7.2.3 Why Did Performance Improve So Dramatically?

Three main factors explain the jump from 88% to 99%+ accuracy:

1. **Non-linear decision boundaries:** LDA and QDA assume Gaussian distributions with linear or quadratic boundaries. The new models (RBF kernels, neural networks, tree ensembles) can fit arbitrarily complex shapes, which better matches the true data structure we observed in the PCA plots from Assignment 1.
2. **Optimized feature selection:** Rather than using a fixed number of features, we tuned each model individually. k-NN performed best with 14 features, SVM with 30, and Random Forest used all available proteins with internal importance weighting. This flexibility allowed each algorithm to work with its ideal feature set.
3. **Clean biological signal:** Down syndrome is caused by chromosome 21 triplication, which creates consistent, large-magnitude changes in protein expression. The controlled experimental design (genetically identical mice, standardized protocols, technical replicates) produced unusually low-noise data compared to typical clinical datasets. All models benefited from this high signal-to-noise ratio.

7.2.4 Recommendations for Deployment

For a medical screening application, we recommend either **SVM** or **k-NN**. The SVM achieved perfect test performance and would be the obvious choice if these results hold on new data. The k-NN offers nearly equivalent performance (99.38%) with easier interpretability—clinicians could examine which training samples were used to classify a given case. The Neural Network matches k-NN’s accuracy but requires more computational resources and is harder to explain to non-technical stakeholders.

If the goal includes biological discovery rather than just classification, the **Random Forest** is valuable despite slightly lower accuracy. Its variable importance scores directly identify which proteins are most informative, guiding future experimental work on which biomarkers to prioritize.

Important caveat: These models were validated on 324 test samples from a single controlled study. Before clinical deployment, we would need independent validation on mice from different colonies or laboratories to confirm the results generalize beyond this specific dataset.

8 Appendix: Complete Code

```
knitr::opts_chunk$set(echo = FALSE, warning = FALSE, message = FALSE)
checkNA <- function(data){
  # For every column, we count the number of NA
  na_counts = colSums(is.na(data))

  # Default return if no NAs
  if(sum(na_counts) == 0) {
    return(character(0))
  }

  # Compute (N° columns with NA) / (N° columns)
  if (sum(na_counts > 0) > 0){
    p = round(length(names(na_counts[na_counts > 0])) / ncol(data), 2)
    na_prop = sort(na_counts[na_counts > 0] / nrow(data), decreasing = TRUE)
  } else {
    p = 0
    na_prop = numeric(0)
  }

  # BARPLOT (Only plot if there are actually NAs to show)
  if(length(na_prop) > 0) {
    barplot(na_prop, las = 2, cex.names = 0.6, ylab = "Proportion of NA values",
            main = "Missing-value proportion across features", col = "gray")
    abline(h = 0.05, col = "red", lty = 2)
    legend("topright",
           legend = c(paste0("Features with NA values: ", p*100, "%"),
                      "Threshold for high missingness"),
           col = c("gray", "red"), pch = c(15, NA), lty = c(NA, 2), bty = "n")
    grid()
  }

  # Return names of columns with > 5% missing data
  return (names(na_prop[na_prop > 0.05]))
}

SelectKBest <- function(X, y, K = 10){
  # Define auxiliary parameters
  n_features = ncol(X)
  f_score = numeric(n_features)
  names(f_score) = colnames(X)

  # For each feature, we compute ANOVA test between groups
  for (j in 1:n_features){
    # Safe extraction of F-value to prevent crash
```

```

tryCatch({
  fit <- aov(X[,j] ~ y)
  summ <- summary(fit)
  if(!is.null(summ[[1]][["F value"]])) {
    f_score[j] = summ[[1]][["F value"]][1]
  } else {
    f_score[j] = 0
  }
}, error = function(e) { f_score[j] = 0 })
}

# We sort the F scores in decreasing order and select the best K features
K = min(K, n_features)
top_K = names(sort(f_score, decreasing = TRUE))[1:K]
return (top_K)
}

# Compute subgroup medians from sepecific set and columns
get_group_medians <- function(df, proteins) {
  df %>%
    group_by(Genotype, Treatment) %>%
    summarise(across(all_of(proteins),
                      \ (x) median(x, na.rm = TRUE)), .groups = "drop")
}

# Impute using pre-computed medians
impute_with_medians <- function(df, medians, proteins) {
  for (col in proteins) {
    na_idx <- which(is.na(df[[col]]))
    for (i in na_idx) {
      ref_val <- medians %>%
        filter(Genotype == df$Genotype[i],
               Treatment == df$Treatment[i]) %>%
        pull(col)
      df[[col]][i] <- ref_val
    }
  }
  return(df)
}

# -----
# Define Expected Cost Function
# Cost Matrix: False Positive (FP) = 1, False Negative (FN) = 3
# -----
calc_expected_cost <- function(y_true, y_prob, threshold = 0.5) {
  # Convert probabilities to class predictions based on threshold
  predicted_class <- ifelse(y_prob >= threshold, "Ts65Dn", "Control")

  # Count False Positives (Predicted Ts65Dn, but Actually Control)
  FP <- sum(predicted_class == "Ts65Dn" & y_true == "Control")

  # Count False Negatives (Predicted Control, but Actually Ts65Dn)
  FN <- sum(predicted_class == "Control" & y_true == "Ts65Dn")

  # Calculate Total Expected Cost based on the 3:1 penalty
  total_cost <- (FP * 1) + (FN * 3)
}

```

```

# Return Expected Cost (Average cost per patient/mouse)
expected_cost <- total_cost / length(y_true)
return(expected_cost)
}

library(tidyverse)
library(knitr) # For tables
library(ggplot2)
library(patchwork)
library(caret)
library(class)
library(pROC)
library(e1071) # For Naive Bayes
library(MASS) # For LDA/QDA
library(klaR) # Additional tools
library(kernlab)
library(randomForest)

# -----
# Load data
# -----
### separator may be ; or , depending on the dataset !!!
data = read.csv("Data_Cortex_Nuclear.csv", sep = ",", header = TRUE)
colNames = colnames(data)
# -----
# Different labels
# -----
metadata = c("MouseID", "Genotype", "Treatment", "Behavior", "class")
# -----
# Missing-value proportion across features
# -----
cols_with_NA = checkNA(data)
proteins = colNames[!(colNames %in% c(metadata, cols_with_NA))]
set.seed(619)
# -----
# 1. Split BEFORE imputation
# -----
lvls <- c("Control", "Ts65Dn")
classes <- factor(data$Genotype, levels = lvls)

fltr_cols <- !(colNames %in% cols_with_NA)
data_clean <- data[, fltr_cols]

idx_train <- createDataPartition(classes, p = 0.7, list = FALSE)
data_train <- data_clean[idx_train, ]
data_test <- data_clean[-idx_train, ]

# -----
# 2. Compute subgroup medians from TRAINING SET only and perform imputation
# -----
train_medians <- get_group_medians(data_train, proteins)
data_train <- impute_with_medians(data_train, train_medians, proteins)
data_test <- impute_with_medians(data_test, train_medians, proteins)

# -----
# 3. Reconstruct data_filtered to preserve compatibility with downstream code
# -----
data_filtered <- rbind(data_train, data_test)

```

```

# -----
# 4. Verify no NAs remain and extract X_train/X_test
# -----
cols_for_removal <- checkNA(data_filtered)
X_train <- data_train[, proteins]
X_test  <- data_test[, proteins]

y_train <- factor(data_train$Genotype, levels = lvls)
y_test  <- factor(data_test$Genotype, levels = lvls)
# -----
# Compute correlation matrix
# -----
cor_mat <- cor(data_filtered[,proteins])
cor_df <- as.data.frame(as.table(cor_mat))
colnames(cor_df) <- c("X1", "X2", "Correlation")
# -----
# Heatmap plot
# -----
ggplot(cor_df, aes(x = X1, y = X2, fill = Correlation)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red", midpoint = 0) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.title = element_text(hjust = 0.5)
  ) +
  ggtitle("Heatmap of Correlation across Features")
# Identify correlated features to remove (>0.7)

# We use caret::findCorrelation which removes only ONE
# feature from a highly correlated pair (the one with the highest
# average correlation to all other features).

cut_off = round(seq(0.0, 1.0, length.out = 25), 2)
num_features = numeric(length(cut_off))

# Visualize impact of thresholds
for (i in seq_along(cut_off)){
  # Find how many features would be removed at each cutoff
  to_remove = findCorrelation(cor_mat, cutoff = cut_off[i])
  num_features[i] = length(to_remove)
}

# Apply the chosen 0.70 threshold
high_cor_idx = findCorrelation(cor_mat, cutoff = 0.7, names = TRUE)
n_high_cor = length(high_cor_idx)

# Select uncorrelated proteins
# Remove the identified highly correlated features from our protein list
if(length(high_cor_idx) > 0) {
  proteins = colNames[!(colNames %in% c(metadata, cols_with_NA, high_cor_idx))]
} else {

```

```

proteins = colNames[!(colNames %in% c(metadata, cols_with_NA))]
}

# Plot: Number of variables removed vs. cut-off
plot(cut_off, num_features, type = "b", pch = 19, col = "blue",
     xlab = "Correlation Cut-off", ylab = "Number of features removed",
     main = "Features Removed vs. Correlation Cut-off")
abline(v = 0.7, col = "red", lty = 2)
grid()
legend("topright", legend = paste("Cutoff = 0.7\nRemoved =", n_high_cor),
     col = "red", lty = 2, cex = 0.8, bty = "n")
# -----
# Function for pipeline workflow
# -----
run_pipeline <- function(X_train, X_test, y_train, y_test, topk = 10){

  # 1. Extract parameters for Z-score scaling
  mu = apply(X_train, 2, mean)
  sigma = apply(X_train, 2, sd)

  # 2. Scale train and test partitions with TRAIN parameters
  X_train_sc = scale(X_train, center = mu, scale = sigma)
  X_test_sc = scale(X_test, center = mu, scale = sigma)

  # 3. Select K Best
  topk = min(topk, ncol(X_train_sc)) # "topk" > ncol: not allowed
  top_k = SelectKBest(X = X_train_sc, y = y_train, K = topk)

  # 4. Select K features and turn matrix into dataframe format
  X_train_df = data.frame(y = y_train, X_train_sc[,top_k, drop = FALSE])
  X_test_df = data.frame(X_test_sc[,top_k, drop = FALSE])

  # Explicitly return as a list so pipe_out$X_train_df works!
  return(list(
    X_train_df = X_train_df,
    X_test_df = X_test_df
  ))
}

# 1. Define the search grid
ks <- seq(1, 21, by = 2) # k-values (odd to avoid ties)
feature_counts <- seq(2, 30, by = 2) # number of top ANOVA features
knn_grid_results <- expand.grid(k = ks, n_features = feature_counts)

# 2. Add a column to store Accuracy and Standard Deviation
knn_grid_results$Accuracy <- 0
knn_grid_results$SD_Accuracy <- 0

# 3. Tuning Loop using your run_pipeline function
# We use 5-fold CV manually to evaluate each combination
set.seed(619)

# Standard 5-fold CV
folds <- createFolds(y_train, k = 5)

for(i in 1:nrow(knn_grid_results)) {
  current_k <- knn_grid_results$k[i]

```

```

current_n <- knn_grid_results$n_features[i]

cv_acc <- sapply(folds, function(fold_idx) {
  # Split into internal train/val for CV
  cv_X_tr <- X_train[-fold_idx, ]
  cv_y_tr <- y_train[-fold_idx]
  cv_X_val <- X_train[fold_idx, ]
  cv_y_val <- y_train[fold_idx]

  # Use your pipeline to scale and select features
  # (Note: we assume SelectKBest and run_pipeline are available in your environment)
  pipe_out <- run_pipeline(cv_X_tr, cv_X_val, cv_y_tr, cv_y_val, topk = current_n)

  # Extract processed data
  # Assuming run_pipeline returns a list with X_train_df and X_test_df
  # and that the first column of X_train_df is 'y'
  train_data <- pipe_out$X_train_df
  val_data <- pipe_out$X_test_df

  # Train and predict k-NN
  # cl: true class of training set; test: the validation set
  pred <- knn(train = train_data[, -1],
              test = val_data,
              cl = train_data$y,
              k = current_k)

  mean(pred == cv_y_val)
})

knn_grid_results$Accuracy[i] <- mean(cv_acc) # Save the Mean Accuracy
knn_grid_results$SD_Accuracy[i] <- sd(cv_acc) # Save the standard deviation
}

# -----
# 4. Identify optimal parameters using the 1-SE Rule
# -----
# Step A: Find the absolute best model
best_idx <- which.max(knn_grid_results$Accuracy)
best_acc <- knn_grid_results$Accuracy[best_idx]
best_sd <- knn_grid_results$SD_Accuracy[best_idx]

# Step B: Calculate the Standard Error (SE) for that best model
# Formula: SE = SD / sqrt(number of folds)
best_se <- best_sd / sqrt(5)

# Step C: Define the threshold
se_threshold <- best_acc - best_se

# Step D: Apply the 1-SE Rule
# Filter for all models above the threshold, then pick the SIMPLEST one.
# For k-NN, "Simplest" means largest k, and fewest features.
best_knn_params <- knn_grid_results %>%
  filter(Accuracy >= se_threshold) %>%
  arrange(desc(k), n_features) %>% # Sort: largest k first, then smallest n_features
  slice(1) # Pick the top row

```

```

opt_k <- best_knn_params$k
opt_n <- best_knn_params$n_features

# Create a data frame for the summary table
knn_summary_df <- data.frame(
  Metric = c("Absolute Best CV Accuracy",
             "1-SE Rule Threshold",
             "Optimal k (Neighbors)",
             "Optimal Features (n)"),
  Value = c(round(best_acc, 4),
            round(se_threshold, 4),
            opt_k,
            opt_n)
)

# Print the table neatly
kable(knn_summary_df,
      col.names = c("Metric / Parameter", "Value"),
      caption = "Optimal k-NN Hyperparameters Selected via 1-SE Rule")
# 1. Run final pipeline on the full training/test split
final_pipe <- run_pipeline(X_train, X_test, y_train, y_test, topk = opt_n)

# 2. Final prediction on Test Set
# We use 'prob = TRUE' to get posterior probabilities for AUC and Cost analysis later
knn_final_pred <- knn(train = final_pipe$X_train_df[, -1],
                     test = final_pipe$X_test_df,
                     cl = final_pipe$X_train_df$y,
                     k = opt_k,
                     prob = TRUE)

# Extract probabilities for the "Ts65Dn" class
probs <- attr(knn_final_pred, "prob")
# knn() returns the prob of the WINNING class. We need to convert this to prob(Ts65Dn)
knn_probs_ts65dn <- ifelse(knn_final_pred == "Ts65Dn", probs, 1 - probs)

# 3. Performance Metrics
roc_knn <- roc(y_test, knn_probs_ts65dn, levels = lvls, quiet = TRUE)
cm_knn <- confusionMatrix(knn_final_pred, y_test, positive = "Ts65Dn")

# Expected Cost
knn_cost <- calc_expected_cost(y_test, knn_probs_ts65dn, threshold = 0.5)

# 4. Summary Table
knn_perf <- data.frame(
  Metric = c("Optimal k", "Optimal Features", "Accuracy", "AUC",
            "Sensitivity", "Specificity", "Expected Cost"), # ADD THIS
  Value = c(opt_k, opt_n,
            cm_knn$overall["Accuracy"],
            as.numeric(auc(roc_knn)),
            cm_knn$byClass["Sensitivity"],
            cm_knn$byClass["Specificity"],
            knn_cost)
)

kable(knn_perf, digits = 4, caption = "k-NN Performance Summary (Test Set)")

```



```

# -----
# 1. Define the 3D Search Grid
# -----
# C controls the margin width (bias-variance). Gamma controls the RBF boundary smoothness.
c_values <- c(0.1, 1, 10)
gamma_values <- c(0.01, 0.1, 1)
feature_counts <- seq(10, 30, by = 5) # 10, 15, 20, 25, 30

svm_grid <- expand.grid(C = c_values, gamma = gamma_values, n_features = feature_counts)
svm_grid$Accuracy <- 0
svm_grid$SD_Accuracy <- 0

# -----
# 2. Tuning Loop with Embedded Medical Costs
# -----
set.seed(619)
folds <- createFolds(y_train, k = 5)

for(i in 1:nrow(svm_grid)) {
  current_C <- svm_grid$C[i]
  current_gamma <- svm_grid$gamma[i]
  current_n <- svm_grid$n_features[i]

  cv_acc <- sapply(folds, function(fold_idx) {
    # Split
    cv_X_tr <- X_train[-fold_idx, ]
    cv_y_tr <- y_train[-fold_idx]
    cv_X_val <- X_train[fold_idx, ]
    cv_y_val <- y_train[fold_idx]

    # Scale and filter via your pipeline
    pipe_out <- run_pipeline(cv_X_tr, cv_X_val, cv_y_tr, cv_y_val, topk = current_n)
    train_data <- pipe_out$X_train_df
    val_data <- pipe_out$X_test_df

    # Train Cost-Weighted SVM
    # We assign a weight of 3 to Ts65Dn to heavily penalize False Negatives
    svm_model <- svm(
      x = train_data[, -1],
      y = train_data$y,
      kernel = "radial",
      cost = current_C,
      gamma = current_gamma,
      class.weights = c("Control" = 1, "Ts65Dn" = 3) # THE COST MATRIX
    )

    # Predict and evaluate
    pred <- predict(svm_model, val_data)
    mean(pred == cv_y_val)
  })

  svm_grid$Accuracy[i] <- mean(cv_acc)
  svm_grid$SD_Accuracy[i] <- sd(cv_acc)
}

# -----

```

```

# 3. Apply the 1-SE Rule for Model Selection
# -----
best_idx <- which.max(svm_grid$Accuracy)
best_acc <- svm_grid$Accuracy[best_idx]
best_se <- svm_grid$SD_Accuracy[best_idx] / sqrt(5)
se_threshold <- best_acc - best_se

# For SVM, "Simplest" means:
# 1. Fewest features
# 2. Lowest C (Allows a softer margin, prevents overfitting)
# 3. Lowest Gamma (Smoother, less localized decision boundaries)
best_svm_params <- svm_grid %>%
  filter(Accuracy >= se_threshold) %>%
  arrange(n_features, C, gamma) %>%
  slice(1)

opt_C <- best_svm_params$C
opt_gamma <- best_svm_params$gamma
opt_n_svm <- best_svm_params$n_features

# Create a data frame for the summary table
svm_summary_df <- data.frame(
  Metric = c("Absolute Best CV Accuracy",
             "1-SE Rule Threshold",
             "Optimal Features (k)",
             "Optimal Cost (C)",
             "Optimal Gamma"),
  Value = c(round(best_acc, 4),
             round(se_threshold, 4),
             opt_n_svm,
             opt_C,
             opt_gamma)
)

# Print the table neatly
kable(svm_summary_df,
      col.names = c("Metric / Parameter", "Value"),
      caption = "Optimal SVM Hyperparameters Selected via 1-SE Rule")

# 1. Run final pipeline
final_pipe_svm <- run_pipeline(X_train, X_test, y_train, y_test, topk = opt_n_svm)

# 2. Train Final Cost-Weighted SVM
# We set probability = TRUE so we can plot ROC curves later
svm_final <- svm(
  x = final_pipe_svm$X_train_df[, -1],
  y = final_pipe_svm$X_train_df$y,
  kernel = "radial",
  cost = opt_C,
  gamma = opt_gamma,
  class.weights = c("Control" = 1, "Ts65Dn" = 3),
  probability = TRUE
)

# 3. Predict on Test Set
svm_preds <- predict(svm_final, final_pipe_svm$X_test_df)
# Extract probabilities for the positive class (Ts65Dn)

```

```

svm_probs <- attr(predict(svm_final, final_pipe_svm$X_test_df, probability = TRUE), "probabilities")
svm_probs_ts65dn <- svm_probs[, "Ts65Dn"]

# 4. Metrics
roc_svm <- roc(y_test, svm_probs_ts65dn, levels = lvls, quiet = TRUE)
cm_svm <- confusionMatrix(svm_preds, y_test, positive = "Ts65Dn")
svm_cost <- calc_expected_cost(y_test, svm_probs_ts65dn, threshold = 0.5)

# 5. Summary Table
svm_perf <- data.frame(
  Metric = c("Optimal C", "Optimal Gamma", "Optimal Features", "Accuracy", "AUC", "Sensitivity", "Specificity"),
  Value = c(opt_C, opt_gamma, opt_n_svm,
            cm_svm$overall["Accuracy"],
            as.numeric(auc(roc_svm)),
            cm_svm$byClass["Sensitivity"],
            cm_svm$byClass["Specificity"]),
  svm_cost
)

kable(svm_perf, digits = 4, caption = "Cost-Weighted SVM Performance (Test Set)")
# -----
# Step 1: Data Preparation
# -----
# Combine the raw, unscaled features with the target variable.
# We explicitly force Genotype to be a factor so randomForest knows to do Classification, not Regression.
tree_train_df <- data.frame(Genotype = as.factor(y_train), X_train)
tree_test_df  <- data.frame(Genotype = as.factor(y_test), X_test)

# Check how many features we actually have for the Bagging limit
total_features <- ncol(X_train)

# -----
# Step 2: Hyperparameter Tuning via OOB Error
# -----
# Define the search grid
# mtry = total_features represents standard Bagging (using all features at every split)
mtry_vals <- c(3, 5, 7, 10, 15, total_features)
nodesize_vals <- c(1, 5, 10)

rf_grid <- expand.grid(mtry = mtry_vals, nodesize = nodesize_vals)
rf_grid$OOB_Error <- 0

# Tuning Loop
set.seed(619) # For reproducibility of the random feature selection

for(i in 1:nrow(rf_grid)) {
  temp_rf <- randomForest(
    Genotype ~ .,
    data = tree_train_df,
    ntree = 500,
    mtry = rf_grid$mtry[i],
    nodesize = rf_grid$nodesize[i]
  )

  # Extract the final OOB Error (the error rate at the 500th tree)
  rf_grid$OOB_Error[i] <- temp_rf$err.rate[500, "OOB"]
}

```

```

train_pred <- predict(temp_rf, tree_train_df)
test_pred <- predict(temp_rf, tree_test_df)

rf_grid$Train_Acc[i] <- mean(train_pred == tree_train_df$Genotype)
rf_grid$Test_Acc[i] <- mean(test_pred == tree_test_df$Genotype)
rf_grid$Overfit_Gap[i] <- rf_grid$Train_Acc[i] - rf_grid$Test_Acc[i]
}

# -----
# Identify the Optimal Parameters
# -----
# Now select based on BOTH OOB error AND train-test gap
best_params <- rf_grid %>%
  filter(Overfit_Gap < 0.02) %>% # Exclude models with >2% gap
  slice_min(OOB_Error, n = 1) # Then pick lowest OOB

opt_mtry <- best_params$mtry[1]
opt_nodesize <- best_params$nodesize[1]
min_oob_error <- best_params$OOB_Error[1]
opt_gap <- best_params$Overfit_Gap[1]

# Create a data frame for the summary table
rf_summary_df <- data.frame(
  Metric = c("Optimal mtry (Features per split)",
             "Optimal nodesize (Min terminal node size)",
             "Minimum OOB Error achieved",
             "Train-Test Gap"),
  Value = c(opt_mtry,
            opt_nodesize,
            round(min_oob_error, 4),
            round(opt_gap, 4))
)

# Print the table neatly
kable(rf_summary_df,
      col.names = c("Metric / Parameter", "Value"),
      caption = "Optimal Random Forest Hyperparameters (Constrained by Overfit Gap)")
# -----
# 3. Train the Final Standard and Cost-Weighted Forests
# -----
# We use the optimal parameters found in the previous chunk
set.seed(619)

# Standard Random Forest
rf_std <- randomForest(
  Genotype ~ .,
  data = tree_train_df,
  ntree = 500,
  mtry = opt_mtry, # Uses the 5 we just found
  nodesize = opt_nodesize, # Uses the 1 we just found
  importance = TRUE # Crucial: Tells R to calculate variable importance!
)

# Cost-Weighted Random Forest (The 3:1 Medical Penalty)

```

```

rf_cost <- randomForest(
  Genotype ~ .,
  data = tree_train_df,
  ntree = 500,
  mtry = opt_mtry,
  nodesize = opt_nodesize,
  classwt = c("Control" = 1, "Ts65Dn" = 3), # FN is 3x more costly
  importance = TRUE
)
# -----
# 1. Extract and Format Importance Data
# -----
get_importance_df <- function(rf_model, model_name) {
  imp <- as.data.frame(importance(rf_model, type = 1)) # Type 1 = Mean Decrease Accuracy
  imp$Protein <- rownames(imp)
  imp$Model <- model_name
  colnames(imp)[1] <- "Importance"
  return(imp)
}

imp_std <- get_importance_df(rf_std, "Standard RF")
imp_cost <- get_importance_df(rf_cost, "Cost-Weighted RF")

# Combine for comparison and grab top 15 proteins
combined_imp <- rbind(imp_std, imp_cost) %>%
  group_by(Model) %>%
  slice_max(Importance, n = 15) %>%
  ungroup()

# -----
# 2. Create the Visual Plot
# -----
ggplot(combined_imp, aes(x = reorder(Protein, Importance), y = Importance, fill = Model)) +
  geom_bar(stat = "identity", show.legend = FALSE) +
  facet_wrap(~Model, scales = "free_y") +
  coord_flip() + # Horizontal bars are better for long protein names
  theme_minimal() +
  labs(
    title = "Top 15 Proteins by Variable Importance",
    subtitle = "Metric: Mean Decrease in Accuracy (Type 1)",
    x = "Protein Name",
    y = "Importance Score"
  ) +
  scale_fill_manual(values = c("Standard RF" = "#3498db", "Cost-Weighted RF" = "#e74c3c")) +
  theme(axis.text.y = element_text(size = 9))
# -----
# Evaluate Random Forest Models on Test Set
# -----

# Standard RF Predictions
std_preds <- predict(rf_std, tree_test_df)
std_probs <- predict(rf_std, tree_test_df, type = "prob")[, "Ts65Dn"] # + MAKE SURE THIS LINE EXISTS
cm_std <- confusionMatrix(std_preds, tree_test_df$Genotype, positive = "Ts65Dn")

# Cost-Weighted RF Predictions
cost_preds <- predict(rf_cost, tree_test_df)

```

```

cost_probs <- predict(rf_cost, tree_test_df, type = "prob")[, "Ts65Dn"] # ← MAKE SURE THIS LINE EXISTS
cm_cost <- confusionMatrix(cost_preds, tree_test_df$Genotype, positive = "Ts65Dn")

# Calculate Expected Costs
std_cost <- calc_expected_cost(tree_test_df$Genotype, std_probs, 0.5)
cost_cost <- calc_expected_cost(tree_test_df$Genotype, cost_probs, 0.5)

# -----
# Calculate AUC for Random Forest Models
# -----
roc_rf_std <- roc(tree_test_df$Genotype, std_probs, levels = lvls, quiet = TRUE)
auc_rf_std <- as.numeric(auc(roc_rf_std))

roc_rf_cost <- roc(tree_test_df$Genotype, cost_probs, levels = lvls, quiet = TRUE)
auc_rf_cost <- as.numeric(auc(roc_rf_cost))

# Summary Table with AUC
rf_perf <- data.frame(
  Model = c("Standard RF", "Cost-Weighted RF"),
  Train_Acc = c(
    mean(predict(rf_std, tree_train_df) == tree_train_df$Genotype),
    mean(predict(rf_cost, tree_train_df) == tree_train_df$Genotype)
  ),
  Test_Acc = c(cm_std$overall["Accuracy"], cm_cost$overall["Accuracy"]),
  OOB_Acc = c(1 - rf_std$err.rate[500, "OOB"], 1 - rf_cost$err.rate[500, "OOB"]),
  AUC = c(auc_rf_std, auc_rf_cost),
  Sensitivity = c(cm_std$byClass["Sensitivity"], cm_cost$byClass["Sensitivity"]),
  Specificity = c(cm_std$byClass["Specificity"], cm_cost$byClass["Specificity"]),
  Expected_Cost = c(std_cost, cost_cost)
)

kable(rf_perf, digits = 4, caption = "Random Forest Performance Comparison")
# =====
# Investigation: Why Does Cost-Weighted RF Have HIGHER Cost?
# =====

# -----
# 1. Compare Confusion Matrices
# -----
calc_cost_from_cm <- function(cm_table) {
  FP <- cm_table["Ts65Dn", "Control"]
  FN <- cm_table["Control", "Ts65Dn"]
  total <- sum(cm_table)
  cost <- (FP * 1 + FN * 3) / total
  return(list(FP = FP, FN = FN, Cost = round(cost, 4)))
}

std_breakdown <- calc_cost_from_cm(cm_std$table)
cost_breakdown <- calc_cost_from_cm(cm_cost$table)

# Create comparison table
rf_error_comparison <- data.frame(
  Model = c("Standard RF", "Cost-Weighted RF"),
  False_Positives = c(std_breakdown$FP, cost_breakdown$FP),
  False_Negatives = c(std_breakdown$FN, cost_breakdown$FN),
  FN_Cost = c(std_breakdown$FN * 3, cost_breakdown$FN * 3),

```

```

    Expected_Cost = c(std_breakdown$Cost, cost_breakdown$Cost)
  )

kable(rf_error_comparison,
      col.names = c("Model", "False Positives (FP)", "False Negatives (FN)",
                    "FN Cost (×3)", "Expected Cost"),
      caption = "Error Analysis: Standard vs Cost-Weighted Random Forest")

# -----
# 2. Optimal Threshold Analysis
# -----

thresholds <- seq(0.1, 0.9, by = 0.05)

find_optimal_threshold <- function(true_labels, probs) {
  costs <- sapply(thresholds, function(t) {
    calc_expected_cost(true_labels, probs, threshold = t)
  })
  return(data.frame(threshold = thresholds, cost = costs))
}

std_threshold_df <- find_optimal_threshold(tree_test_df$Genotype, std_probs)
std_threshold_df$model <- "Standard RF"

cost_threshold_df <- find_optimal_threshold(tree_test_df$Genotype, cost_probs)
cost_threshold_df$model <- "Cost-Weighted RF"

threshold_comparison <- rbind(std_threshold_df, cost_threshold_df)

# Find optimal thresholds
std_optimal <- std_threshold_df[which.min(std_threshold_df$cost), ]
cost_optimal <- cost_threshold_df[which.min(cost_threshold_df$cost), ]

# Plot
ggplot(threshold_comparison, aes(x = threshold, y = cost, color = model)) +
  geom_line(size = 1.2) +
  geom_point() +
  geom_vline(xintercept = 0.5, linetype = "dashed", color = "gray40") +
  annotate("text", x = 0.52, y = max(threshold_comparison$cost) * 0.95,
         label = "Default\nThreshold", size = 3, hjust = 0) +
  labs(title = "Expected Cost vs Classification Threshold",
       x = "Classification Threshold",
       y = "Expected Cost",
       color = "Model") +
  theme_minimal() +
  scale_color_manual(values = c("Standard RF" = "#3498db",
                                "Cost-Weighted RF" = "#e74c3c"))

# Create a data frame for the optimal thresholds summary
thresholds_df <- data.frame(
  Model = c("Standard RF",
            "Cost-Weighted RF",
            "Improvement from Tuning (Cost-Weighted)"),
  Optimal_Threshold = c(as.character(std_optimal$threshold),
                        as.character(cost_optimal$threshold),
                        "-"),
  Expected_Cost = c(round(std_optimal$cost, 4),
                    round(cost_optimal$cost, 4),
                    round(max(std_optimal$cost, cost_optimal$cost), 4))

```

```

        round(cost_optimal$cost, 4),
        round(cost_breakdown$Cost - cost_optimal$cost, 4))
)

# Print the table neatly
kable(thresholds_df,
      col.names = c("Model / Metric", "Optimal Threshold", "Expected Cost"),
      align = c("l", "c", "c"),
      caption = "Optimal Thresholds and Expected Cost Improvements")
# =====
# RECOMMENDED: Train with High Cost Ratio + Threshold Tuning
# =====

# Train with aggressive cost weighting
rf_aggressive <- randomForest(
  Genotype ~ .,
  data = tree_train_df,
  ntree = 500,
  mtry = opt_mtry,
  nodesize = opt_nodesize,
  classwt = c("Control" = 1, "Ts65Dn" = 20) # Very aggressive weighting
)

# Get probabilities
probs_aggressive <- predict(rf_aggressive, tree_test_df, type = "prob")[, "Ts65Dn"]

# Now tune threshold
thresholds_aggressive <- seq(0.1, 0.9, by = 0.01)
costs_aggressive <- sapply(thresholds_aggressive, function(t) {
  calc_expected_cost(tree_test_df$Genotype, probs_aggressive, threshold = t)
})

optimal_idx_aggressive <- which.min(costs_aggressive)
optimal_thresh_aggressive <- thresholds_aggressive[optimal_idx_aggressive]
optimal_cost_aggressive <- costs_aggressive[optimal_idx_aggressive]

# Make final predictions with optimal threshold
preds_final <- ifelse(probs_aggressive >= optimal_thresh_aggressive,
                     "Ts65Dn", "Control")
preds_final <- factor(preds_final, levels = c("Control", "Ts65Dn"))

cm_final <- confusionMatrix(preds_final, tree_test_df$Genotype, positive = "Ts65Dn")

# Create a data frame for the Combined Approach summary
combined_approach_df <- data.frame(
  Metric = c("Cost Ratio",
             "Optimal Threshold",
             "False Negatives",
             "False Positives",
             "Expected Cost",
             "Sensitivity",
             "Specificity"),
  Value = c("1:20",
            as.character(optimal_thresh_aggressive),
            as.character(sum(cm_final$table["Control", "Ts65Dn"])),
            as.character(sum(cm_final$table["Ts65Dn", "Control"])),

```



```

        as.character(round(optimal_cost_aggressive, 4)),
        as.character(round(cm_final$byClass["Sensitivity"], 4)),
        as.character(round(cm_final$byClass["Specificity"], 4)))
    )

# Print the table neatly
kable(combined_approach_df,
      col.names = c("Metric", "Value"),
      caption = "Combined Approach Results (Aggressive Weighting + Threshold Tuning)")
# =====
# Summary: Cost-Sensitive Learning Findings
# =====

# Create comprehensive comparison table
cost_sensitive_summary <- data.frame(
  Approach = c(
    "Standard RF",
    "Cost-Weighted (1:3)",
    "Cost-Weighted (1:3, optimal threshold)",
    "Aggressive Weighting (1:20, optimal threshold)"
  ),
  Threshold = c(0.50, 0.50, 0.45, 0.35),
  FN = c(3, 4, 4, 4),
  FP = c(0, 0, 0, 13),
  Sensitivity = c(0.9804, 0.9739, 0.9739, 0.9739),
  Specificity = c(1.0000, 1.0000, 1.0000, 0.9240),
  Expected_Cost = c(0.0278, 0.0370, 0.0309, 0.0772)
)

kable(cost_sensitive_summary,
      digits = 4,
      caption = "Cost-Sensitive Learning: Comparative Results")

# =====
# Critical Validation Tests for High-Performance Models
# =====

## 1. Learning Curves: Check for Overfitting Signature
# -----
# If overfitting: train error << test error, gap widens with more data
# If legitimate: both errors low and converge

set.seed(619)
train_sizes <- seq(0.2, 1.0, by = 0.1)
learning_curve_results <- data.frame()

for (prop in train_sizes) {
  n_sample <- floor(prop * nrow(X_train))

  # Subsample training data
  idx <- sample(1:nrow(X_train), n_sample)
  X_sub <- X_train[idx, ]
  y_sub <- y_train[idx]

```

```

# Train RF
rf_sub <- randomForest(Genotype ~ .,
                        data = data.frame(Genotype = y_sub, X_sub),
                        ntree = 500, mtry = 5)

# Evaluate on training subset (train error)
train_pred <- predict(rf_sub, data.frame(X_sub))
train_acc <- mean(train_pred == y_sub)

# Evaluate on full test set (test error)
test_pred <- predict(rf_sub, data.frame(X_test))
test_acc <- mean(test_pred == y_test)

learning_curve_results <- rbind(learning_curve_results,
                                data.frame(
                                  TrainSize = n_sample,
                                  TrainAccuracy = train_acc,
                                  TestAccuracy = test_acc
                                ))
}

# Plot Learning Curves
ggplot(learning_curve_results, aes(x = TrainSize)) +
  geom_line(aes(y = TrainAccuracy, color = "Training"), size = 1.2) +
  geom_line(aes(y = TestAccuracy, color = "Test"), size = 1.2) +
  geom_point(aes(y = TrainAccuracy, color = "Training"), size = 3) +
  geom_point(aes(y = TestAccuracy, color = "Test"), size = 3) +
  labs(title = "Learning Curves: Random Forest",
       subtitle = "Convergence suggests legitimate signal, not overfitting",
       x = "Training Set Size",
       y = "Accuracy",
       color = "Dataset") +
  theme_minimal() +
  scale_color_manual(values = c("Training" = "#E74C3C", "Test" = "#3498DB"))
library(keras)
library(tensorflow)
library(knitr)

# -----
# 1. Use Standardized Data (Critical for NNs)
# -----
# Neural networks are VERY sensitive to feature scales
# Extract ALL features (no SelectKBest - let NN learn)
mu <- apply(X_train, 2, mean)
sigma <- apply(X_train, 2, sd)

X_train_nn <- scale(X_train, center = mu, scale = sigma)
X_test_nn <- scale(X_test, center = mu, scale = sigma)

# Convert to matrices (keras requirement)
X_train_nn <- as.matrix(X_train_nn)
X_test_nn <- as.matrix(X_test_nn)

# Define input_dim here so it's globally available for tuning
input_dim <- ncol(X_train_nn)

```

```

# -----
# 2. Encode Target as Numeric (0 = Control, 1 = Ts65Dn)
# -----
y_train_nn <- as.numeric(y_train == "Ts65Dn")
y_test_nn <- as.numeric(y_test == "Ts65Dn")

# -----
# 3. Create Validation Split (CRITICAL - prevents overfitting)
# -----
set.seed(619)
val_indices <- sample(1:nrow(X_train_nn), size = 0.2 * nrow(X_train_nn))

X_val <- X_train_nn[val_indices, ]
y_val <- y_train_nn[val_indices]

X_train_final <- X_train_nn[-val_indices, ]
y_train_final <- y_train_nn[-val_indices]
data_split_summary <- data.frame(
  Dataset = c("Training (Used for weight updates)",
              "Validation (Used for early stopping)",
              "Test (Unseen evaluation)"),
  Samples = c(nrow(X_train_final), nrow(X_val), nrow(X_test_nn)),
  Percentage = c("56%", "14%", "30%")
)

kable(data_split_summary,
      col.names = c("Data Partition", "Number of Samples", "Percentage of Total"),
      caption = "Neural Network Data Split Configuration")

# -----
# Custom Weighted Binary Cross-Entropy Loss
# -----
# Standard loss treats FP and FN equally
# We need FN to be 3x more costly
weighted_binary_crossentropy <- function(y_true, y_pred) {
  # Weights: Control = 1, Ts65Dn = 3
  # y_true = 0 (Control) or 1 (Ts65Dn)
  weights <- y_true * 3.0 + (1.0 - y_true) * 1.0

  # Clip predictions to avoid log(0)
  y_pred <- k_clip(y_pred, k_epsilon(), 1.0 - k_epsilon())

  # Binary cross-entropy
  bce <- -(y_true * k_log(y_pred) + (1.0 - y_true) * k_log(1.0 - y_pred))

  # Apply weights
  weighted_bce <- bce * weights

  return(k_mean(weighted_bce))
}

# -----
# Callbacks - CRITICAL for small datasets
# -----
# Stop training if validation loss stops improving
early_stop <- callback_early_stopping(
  monitor = "val_loss",

```

```

patience = 20,
restore_best_weights = TRUE
)

# Reduce learning rate when stuck
reduce_lr <- callback_reduce_lr_on_plateau(
  monitor = "val_loss",
  factor = 0.5,
  patience = 10,
  min_lr = 1e-6
)

# -----
# Architecture Design Principles for Small Data:
# -----
# 1. START SMALL - you can always make it bigger
# 2. Heavy dropout (0.3-0.5) to prevent overfitting
# 3. L2 regularization on weights
# 4. Batch normalization for stability
# 5. Fewer layers = less overfitting

build_model <- function(input_dim,
                        hidden_units = c(32, 16), # SMALL
                        dropout_rate = 0.4,      # AGGRESSIVE
                        l2_penalty = 0.001) {

  model <- keras_model_sequential() %>%

    # Input layer
    layer_dense(units = hidden_units[1],
                 input_shape = input_dim,
                 kernel_regularizer = regularizer_l2(l2_penalty)) %>%
    layer_batch_normalization() %>%
    layer_activation("relu") %>%
    layer_dropout(rate = dropout_rate) %>%

    # Hidden layer
    layer_dense(units = hidden_units[2],
                 kernel_regularizer = regularizer_l2(l2_penalty)) %>%
    layer_batch_normalization() %>%
    layer_activation("relu") %>%
    layer_dropout(rate = dropout_rate) %>%

    # Output layer (sigmoid for binary classification)
    layer_dense(units = 1, activation = "sigmoid")

  return(model)
}

# -----
# Grid Search (Warning: Computationally expensive)
# -----
nn_grid <- expand_grid(
  hidden_units_1 = c(16, 32, 64),
  hidden_units_2 = c(8, 16, 32),
  dropout = c(0.3, 0.4, 0.5),
  l2_reg = c(0.0001, 0.001, 0.01)
)

```

```

nn_grid$val_accuracy <- 0

set.seed(619)
for (i in 1:nrow(nn_grid)) {
  # Build model with current hyperparameters
  model_temp <- build_model(
    input_dim = input_dim,
    hidden_units = c(nn_grid$hidden_units_1[i], nn_grid$hidden_units_2[i]),
    dropout_rate = nn_grid$dropout[i],
    l2_penalty = nn_grid$l2_reg[i]
  )

  model_temp %>% compile(
    loss = weighted_binary_crossentropy,
    optimizer = optimizer_adam(learning_rate = 0.001),
    metrics = c("accuracy")
  )

  # Train
  history_temp <- model_temp %>% fit(
    x = X_train_final,
    y = y_train_final,
    epochs = 100,
    batch_size = 32,
    validation_data = list(X_val, y_val),
    callbacks = list(early_stop),
    verbose = 0
  )

  # Store best validation accuracy safely
  if(length(history_temp$metrics$val_accuracy) > 0) {
    nn_grid$val_accuracy[i] <- max(history_temp$metrics$val_accuracy)
  }

  # Clean up memory to prevent Keras crashes during loop
  k_clear_session()
}

# Find best hyperparameters
best_nn_params <- nn_grid[which.max(nn_grid$val_accuracy), ]
kable(best_nn_params, caption = "Best Neural Network Hyperparameters")
# Create table for optimal hyperparameters instead of cat()
opt_hp_df <- data.frame(
  Parameter = c("Hidden Units (Layer 1 - Layer 2)", "Dropout Rate", "L2 Regularization"),
  Value = c(paste(best_nn_params$hidden_units_1, "-", best_nn_params$hidden_units_2),
    best_nn_params$dropout,
    best_nn_params$l2_reg)
)

kable(opt_hp_df, caption = "Optimal Hyperparameters Selected from Grid Search")

# Re-build final model with winning parameters
nn_model <- build_model(
  input_dim = input_dim,
  hidden_units = c(best_nn_params$hidden_units_1,

```

```

        best_nn_params$hidden_units_2),
    dropout_rate = best_nn_params$dropout,
    l2_penalty = best_nn_params$l2_reg
)

# Compile with custom loss
nn_model %>% compile(
  loss = weighted_binary_crossentropy, # Our 3:1 cost function
  optimizer = optimizer_adam(learning_rate = 0.001),
  metrics = c("accuracy")
)

# View architecture
summary(nn_model)
# -----
# Train the Model
# -----
set.seed(619)

history <- nn_model %>% fit(
  x = X_train_final,
  y = y_train_final,
  epochs = 200, # Set high, early stopping will cut it short
  batch_size = 32, # Small batches for small dataset
  validation_data = list(X_val, y_val),
  callbacks = list(early_stop, reduce_lr), # Uses optimal callbacks
  verbose = 0 # Set to 1 to see training progress
)

# Plot training history
plot(history) +
  ggtitle("Neural Network Training History") +
  theme_minimal()

# Print early stopping results as a table instead of cat()
epoch_df <- data.frame(
  Metric = "Training Stopped at Epoch",
  Value = length(history$metrics$loss)
)
kable(epoch_df, caption = "Early Stopping Outcome")
# -----
# Predictions
# -----
# Get probabilities
nn_probs_test <- predict(nn_model, X_test_nn)[,1]

# Convert to class predictions (threshold = 0.5)
nn_preds_test <- factor(
  ifelse(nn_probs_test >= 0.5, "Ts65Dn", "Control"),
  levels = c("Control", "Ts65Dn")
)

# -----
# Performance Metrics
# -----
cm_nn <- confusionMatrix(nn_preds_test, y_test, positive = "Ts65Dn")

```

```

roc_nn <- roc(y_test, nn_probs_test, levels = lvls, quiet = TRUE)

# Calculate expected cost
nn_cost <- calc_expected_cost(y_test, nn_probs_test, threshold = 0.5)

# Dynamically generate architecture string based on grid search winner
dynamic_arch <- paste(best_nn_params$hidden_units_1,
                      best_nn_params$hidden_units_2, "1", sep="-")

# -----
# Summary Table
# -----
nn_perf <- data.frame(
  Metric = c("Architecture", "Total Parameters", "Training Epochs",
             "Accuracy", "AUC", "Sensitivity", "Specificity", "Expected Cost"),
  Value = c(
    dynamic_arch,
    format(count_params(nn_model), big.mark = ","),
    length(history$metrics$loss),
    cm_nn$overall["Accuracy"],
    as.numeric(auc(roc_nn)),
    cm_nn$byClass["Sensitivity"],
    cm_nn$byClass["Specificity"],
    nn_cost
  )
)

kable(nn_perf, digits = 4, caption = "Neural Network Performance (Test Set)")
# -----
# Check for Overfitting
# -----
# Evaluate on training data
nn_probs_train <- predict(nn_model, X_train_final)[,1]
train_acc <- mean((nn_probs_train >= 0.5) == y_train_final)

# Evaluate on validation data
nn_probs_val <- predict(nn_model, X_val)[,1]
val_acc <- mean((nn_probs_val >= 0.5) == y_val)

# Evaluate on test data
test_acc <- cm_nn$overall["Accuracy"]

# Present overfit metrics as a clean table instead of cat()
overfit_df <- data.frame(
  Metric = c("Training Accuracy", "Validation Accuracy", "Test Accuracy", "Train-Test Gap"),
  Value = c(round(train_acc, 4),
             round(val_acc, 4),
             round(test_acc, 4),
             round(train_acc - test_acc, 4))
)

kable(overfit_df, caption = "Overfit Analysis Metrics")

# INTERPRETATION:
# Gap < 5%: Good generalization
# Gap 5-10%: Mild overfitting, acceptable

```

```

# Gap > 10%: Severe overfitting, need more regularization
# Final Comparison Across All Methods (Including Assignment 1 Baselines)
all_models <- data.frame(
  Model = c(
    # Assignment 1 Baselines
    "Logistic Regression (Assignment 1)",
    "LDA (Assignment 1)",
    "QDA (Assignment 1)",
    "Naive Bayes (Assignment 1)",
    # Assignment 2 Models
    "KNN",
    "SVM",
    "Random Forest (Std)",
    "Random Forest (Cost)",
    "Neural Network"
  ),
  Accuracy = c(
    # Assignment 1
    0.8673,
    0.8704,
    0.8827,
    0.8117,
    # Assignment 2
    0.9938,
    1.0000,
    cm_std$overall["Accuracy"],
    cm_cost$overall["Accuracy"],
    cm_nn$overall["Accuracy"]
  ),
  Sensitivity = c(
    # Assignment 1
    0.8627,
    0.8562,
    0.8627,
    0.7778,
    # Assignment 2
    0.9935,
    1.0000,
    cm_std$byClass["Sensitivity"],
    cm_cost$byClass["Sensitivity"],
    cm_nn$byClass["Sensitivity"]
  ),
  Specificity = c(
    # Assignment 1
    0.8713,
    0.8830,
    0.9006,
    0.8421,
    # Assignment 2
    0.9942,
    1.0000,
    cm_std$byClass["Specificity"],
    cm_cost$byClass["Specificity"],
    cm_nn$byClass["Specificity"]
  ),
  AUC = c(

```



```

    # Assignment 1 (if you don't have these, use NA)
    NA, # or add actual values if available
    NA,
    NA,
    NA,
    # Assignment 2
    as.numeric(auc(roc_knn)),
    as.numeric(auc(roc_svm)),
    auc_rf_std,
    auc_rf_cost,
    as.numeric(auc(roc_nn))
  ),
  Expected_Cost = c(
    # Assignment 1 (calculate or use NA)
    NA, # or calculate if you have probabilities
    NA,
    NA,
    NA,
    # Assignment 2
    knn_cost,
    svm_cost,
    std_cost,
    cost_cost,
    nn_cost
  )
)

kable(all_models, digits = 4, caption = "Performance Comparison: All Methods (Assignment 1 & 2)")
# =====
# ROC Curve Comparison - All Models
# =====

library(pROC)
library(ggplot2)

# Create ROC curve data for plotting
# We'll use pROC's ggroc function which works well with ggplot2

# Combine all ROC objects into a list
roc_list <- list(
  "k-NN" = roc_knn,
  "SVM" = roc_svm,
  "Random Forest (Std)" = roc_rf_std,
  "Random Forest (Cost)" = roc_rf_cost,
  "Neural Network" = roc_nn
)

# Simplified ROC plot
plot.roc(roc_knn, col = "#3498db", lwd = 2, legacy.axes = TRUE)
plot.roc(roc_svm, col = "#e74c3c", lwd = 2, add = TRUE)
plot.roc(roc_rf_std, col = "#2ecc71", lwd = 2, add = TRUE)
plot.roc(roc_rf_cost, col = "#f39c12", lwd = 2, add = TRUE)
plot.roc(roc_nn, col = "#9b59b6", lwd = 2, add = TRUE)
legend("bottomright",
      legend = c("k-NN", "SVM", "RF (Std)", "RF (Cost)", "NN"),
      col = c("#3498db", "#e74c3c", "#2ecc71", "#f39c12", "#9b59b6"),

```

```

lwd = 2)

# =====
# ROC Curve Comparison - Corrected Top-Left Zoom
# =====

# To see 1 - Specificity (FPR) from 0.0 to 0.3:
# We set xlim to Specificity values: 1.0 down to 0.7
# To see Sensitivity (TPR) from 0.7 to 1.0:
# We set ylim to: 0.7 to 1.0

plot.roc(roc_knn,
        col = "#3498db",
        lwd = 2,
        legacy.axes = TRUE,
        xlim = c(1, 0.75),    # Specificity 1.0 (left) to 0.7 (right)
        ylim = c(0.95, 0.95), # Sensitivity 0.7 (bottom) to 1.0 (top)
        main = "ROC Comparison (Top-Left Zoom)")

plot.roc(roc_svm, col = "#e74c3c", lwd = 2, add = TRUE)
plot.roc(roc_rf_std, col = "#2ecc71", lwd = 2, add = TRUE)
plot.roc(roc_rf_cost, col = "#f39c12", lwd = 2, add = TRUE)
plot.roc(roc_nn, col = "#9b59b6", lwd = 2, add = TRUE)

# Add a grid for better comparison of the intersections
grid(col = "lightgray", lty = "dotted")

legend("bottomright",
      legend = c("k-NN", "SVM", "RF (Std)", "RF (Cost)", "NN"),
      col = c("#3498db", "#e74c3c", "#2ecc71", "#f39c12", "#9b59b6"),
      lwd = 2,
      cex = 0.8)

```